

SQL – Uma linguagem de banco de dados

Este capítulo descreve alguns aspectos da linguagem SQL (*Structure Query Language*) que é uma linguagem de definição, consulta, modificação de esquemas e atualização de banco de dados relacionais (Elmasri e Navathe, 2018). Foi desenvolvida originalmente nos laboratórios da IBM na década de 1970, em pouco tempo vários fabricantes de banco de dados passaram a adotar o SQL como linguagem em seus produtos e hoje é padronizada pelos comitês ANSI/ISO (Ramakrishnan e Gehrke, 2011).

1 Definição de dados em SQL

SQL utiliza os termos tabela, linha e coluna para relação, tupla e atributo, respectivamente. Os comandos SQL para definição de dados (DDL – *Data Definition Language*) são usados para estruturar o banco de dados, suas tabelas e visões (do inglês, *views*), tabelas virtuais que exibem dados de uma ou mais tabelas reais, sem armazená-la fisicamente (Ramakrishnan e Gehrke, 2011). Esses comandos são: CREATE, ALTER e DROP.

1.1 Comando CREATE

O comando CREATE pode ser utilizado para criação de objetos dentro do banco de dados, como esquemas de banco de dados, tabelas, domínios (tipos de dados), restrições de integridade (como chaves primárias e estrangeiras) e visões.

1.1.1 Comando CREATE TABLE

O comando CREATE TABLE permite criar tabelas no banco de dados. Dentro de cada tabela, deve-se especificar os atributos (colunas) e as restrições. Cada atributo deve possuir um nome, um tipo de dado para especificar seu domínio de valores e, possivelmente, algumas restrições. As restrições de chave, restrição de valores nulos de atributos, integridade de entidade e integridade referencial são então especificadas (Ramakrishnan e Gehrke, 2011). Observe a criação das tabelas DEPARTAMENTO e FUNCIONARIO, conforme apresentado a seguir, adaptado de Elmasri e Navathe (2018):

Exemplo 1 – Tabelas DEPARTAMENTO e FUNCIONARIO

```
CREATE TABLE DEPARTAMENTO (  
    NUMDEP          INT NOT NULL,  
    NOME            VARCHAR(100) NOT NULL,  
    PRIMARY KEY (NUMDEP));  
  
CREATE TABLE FUNCIONARIO (  
    NUMAT           INT,  
    NOME            VARCHAR(100) NOT NULL,  
    SOBRENOME       VARCHAR(100) NOT NULL,  
    DTNASC          DATE,  
    ENDER           VARCHAR(50),  
    SALARIO         DECIMAL(10,2),  
    NSUPER          INT,  
    NDEPTO          INT NOT NULL,  
    PRIMARY KEY (NUMAT),  
    FOREIGN KEY (NSUPER) REFERENCES FUNCIONARIO(NUMAT),  
    FOREIGN KEY (NDEPTO) REFERENCES DEPARTAMENTO(NUMDEP));  
  
CREATE TABLE PROJETO (  
    NUMPROJ         INT NOT NULL,  
    NOME            VARCHAR(100) NOT NULL,  
    NDEPTO          INT NOT NULL,  
    PRIMARY KEY (NPROJ)  
    FOREIGN KEY (NDEPTO) REFERENCES DEPARTAMENTO(NUMDEP));  
  
CREATE TABLE TRABALHA_EM (  
    NUMEMP          INT NOT NULL,  
    NPROJ           INT NOT NULL,  
    PRIMARY KEY (NUMEMP, NPROJ),  
    FOREIGN KEY (NUMEMP) REFERENCES FUNCIONARIO(NUMAT),  
    FOREIGN KEY (NPROJ) REFERENCES PROJETO(NUMPROJ)  
);  
  
CREATE TABLE DEPENDENTE (  
    NUMEMP          INT NOT NULL,  
    NOME            VARCHAR(100) NOT NULL,  
    PARENTESCO      VARCHAR(10),  
    PRIMARY KEY (NUMEMP, NOME),  
    FOREIGN KEY (NUMEMP) REFERENCES FUNCIONARIO(NUMAT)  
);
```

O exemplo 1 apresenta um script de criação da tabela DEPARTAMENTO com os atributos NUMDEP do tipo inteiro e NOMEDEP do tipo varchar, com o atributo NUMDEP como chave primária. Além disso, criou-se a tabela FUNCIONARIO com os atributos NUMAT do tipo inteiro, NOME do tipo varchar, DATNASC do tipo data, ENDER do tipo varchar, SALARIO que é um tipo decimal, NSUPER do tipo inteiro e NDEPTO do tipo inteiro. O NUMAT é chave primária. O NSUPER determina um relacionamento recursivo, fazendo um relacionamento com NUMAT. NDEPTO é chave estrangeira para DEPARTAMENTO. Após isso, especificou-se a tabela PROJETO com os atributos NUMPROJ, NOME do projeto, NDEPTO. Em seguida, definiu-se a tabela TRABALHA_EM, que relaciona o funcionário com o projeto. E a última tabela é a de DEPENDENTE que se relaciona com o número do funcionário.

Como exemplo de dados da tabela, observe o quadro 1.

Quadro 1 – Dados das tabelas DEPARTAMENTO e FUNCIONARIO

DEPARTAMENTO

NUMDEP	NOME
1	Administração
2	Pesquisa
3	Projetos

FUNCIONARIO

NUMAT	NOME	SOBRENOME	DTNASC	ENDER	SALARIO	NSUPER	NDEPTO
1111	Andre	Pereira	12/10/1980	Rua A	R\$ 5000.0	1113	1
1112	Antônio	Camargo	14/11/1987	Rua B	R\$ 2500,0	1113	1
1113	Maria	Souza	15/09/1977	Rua C	R\$ 6000,0	NULL	1

Os tipos de dados disponíveis em SQL (Elmasri e Navathe, 2018) são:

- Cadeias de caracteres de comprimento fixo ou variável. O tipo CHAR(*n*) representa uma cadeia de caracteres de comprimento fixo *n*. Se um atributo possui tipo CHAR(*n*), então, em qualquer linha, o componente deste atributo será uma cadeia de *n* caracteres. VARCHAR(*n*) representa cadeias de até *n* caracteres. Componentes de um atributo deste tipo são cadeias de 0 a *n* caracteres.
- Cadeias de *bits* de comprimento fixo ou variável são análogas às cadeias de caracteres de comprimento fixo e variável; entretanto, os seus valores são cadeias de *bits* ao invés de caracteres. O tipo BIT(*n*) representa cadeias de *bits* de comprimento *n*, enquanto BIT VARYING(*n*) representa cadeias de *bits* de comprimento de 0 até *n*.
- O tipo INT ou INTEGER (estes nomes são sinônimos) representam valores inteiros típicos.
- Número de ponto flutuante pode ser representado em uma variedade de formas. Podemos usar o tipo FLOAT ou REAL (estes nomes são sinônimos) para números de ponto flutuante típico. Uma precisão mais alta pode ser obtida com o tipo DOUBLE PRECISION. SQL também possui tipos que são números reais com um ponto decimal fixo. Por exemplo, DECIMAL(*n*, *d*) permite valores que consistem em *n* dígitos decimais, com o ponto decimal posicionado a *d* dígitos a partir da direita. Então, 1234.56 é um possível valor do tipo DECIMAL(6, 2).
- Datas e horas podem ser representadas pelos tipos DATE e TIME, respectivamente.

SQL permite que os valores sejam NULL (nulo). Um valor nulo indica que o valor é indefinido ou é um campo opcional cujo valor é desprezível. Uma restrição NOT NULL pode ser especificada para evitar que um dado atributo possa assumir o valor NULL. Tal restrição deve ser aplicada para

atributos que fazem parte da chave primária, assim como outros atributos cujos valores não podem ser nulos (Elmasri e Navathe, 2018).

As restrições de chaves e integridade referencial podem ser especificadas na criação da tabela. A cláusula PRIMARY KEY indica um ou mais atributos que compõem a chave primária de uma relação. A cláusula UNIQUE determina os atributos cujos valores devem ser únicos na tabela, ou seja, não devem ter duas linhas da tabela com valores iguais para um atributo UNIQUE. Integridade referencial é especificada por meio da cláusula FOREIGN KEY (Ramakrishnan e Gehrke, 2011).

1.1.2 Comando ALTER TABLE

O comando ALTER TABLE é usado para modificar uma tabela já existente, permitindo adicionar, remover ou alterar colunas, assim como modificar restrições. Para adicionar uma nova coluna à tabela FUNCIONARIO observe o exemplo 2, adaptado de Elmasri e Navathe (2018).

Exemplo 2 – Adicionando nova coluna na tabela FUNCIONARIO

```
ALTER TABLE FUNCIONARIO ADD COLUMN SEXO CHAR;
```

Para modificar a definição de um atributo existente, observe o exemplo seguinte:

Exemplo 3 – Modificando a definição de um atributo existente

```
ALTER TABLE FUNCIONARIO ALTER COLUMN SALARIO SET DEFAULT 1000.00;
```

Para remover um atributo da tabela, observe o próximo exemplo:

Exemplo 4 – Removendo um atributo da tabela

```
ALTER TABLE FUNCIONARIO DROP COLUMN ENDER;
```

1.1.3 Comando DROP TABLE

O comando DROP serve para excluir completamente um objeto do banco de dados. O comando DROP TABLE, portanto, exclui uma tabela. Essa ação é irreversível e apaga todos os dados e a estrutura do objeto. Observe o exemplo 5.

Exemplo 5 – Removendo a tabela FUNCIONARIO

```
DROP TABLE FUNCIONARIO;
```



IMPORTANTE

Para realizar os testes na linguagem SQL, pode-se utilizar uma plataforma de SGBD on-line. O LiveSQL da Oracle (Oracle, 2025) é uma plataforma on-line gratuita que permite executar e testar consultas SQL diretamente no navegador, sem necessidade de instalação.

2 Consultas em SQL

SQL possui um comando básico para realizar consultas no banco de dados, o comando SELECT (Elmasri e Navathe, 2018). Uma consulta SQL segue a estrutura, conforme é apresentado no exemplo 6.

Exemplo 6 – Estrutura do comando SELECT

```
SELECT <lista de colunas>  
FROM <lista de tabelas>  
WHERE <condição>;
```

No exemplo 6, temos o comando SELECT para realizar consultas no banco de dados. A estrutura tem três partes, a primeira parte é SELECT e uma lista de colunas que serão apresentadas como resultado da

consulta. A segunda parte é a cláusula FROM e uma lista de tabelas que são as relações a serem processadas para responder à consulta. A terceira parte consiste na cláusula WHERE e uma condição lógica que identifica as tuplas a serem obtidas na consulta.

Onde:

- **<lista de atributos>** é uma lista de nomes de atributos cujos valores serão obtidos pela consulta.
- **<lista de tabelas>** é uma lista dos nomes das relações requeridas para processar a consulta.
- **<condição>** é uma expressão lógica que identifica as tuplas a serem obtidas pela consulta.

2.1 Consultas simples

Suponha a seguinte consulta: obtenha o nome e o salário dos funcionários cujo salário seja maior que 3000. Um exemplo do comando SELECT pode ser observado no exemplo 7.

Exemplo 7 – Comando SELECT

```
SELECT NOME, SALARIO  
FROM FUNCIONARIO  
WHERE SALARIO > 3000;
```

Essa consulta envolve apenas a relação FUNCIONARIO listada na cláusula FROM. A consulta seleciona tuplas de FUNCIONÁRIO que satisfaçam a condição da cláusula WHERE, então projeta o resultado sobre os atributos NOME e SALARIO, listados na cláusula SELECT.

Para selecionar todas as tuplas da tabela e todos os seus atributos utilize o comando SELECT, listado no exemplo 8.

Exemplo 8 – Comando SELECT com todas as tuplas

```
SELECT *  
FROM FUNCIONARIO;
```

Essa consulta envolve apenas a relação FUNCIONARIO listada na cláusula FROM. Para trazer todas as tuplas da relação, não é preciso utilizar a cláusula WHERE. O símbolo asterisco (*) utilizado na instrução indica que a consulta deve trazer todos os atributos no resultado.

2.2 Consultas com junção de tabela

A demonstração seguinte envolve uma consulta que utiliza o relacionamento entre duas tabelas. Deseja-se listar o nome dos funcionários e os departamentos em que eles trabalham, conforme exemplo 9.

Exemplo 9 – Consulta com junção de duas tabelas

```
SELECT FUNCIONARIO.NOME, DEPARTAMENTO.NOME  
FROM FUNCIONARIO, DEPARTAMENTO  
WHERE FUNCIONARIO.NDEPTO = DEPARTAMENTO.NUMDEP;
```

Na consulta apresentada no exemplo 9, a cláusula WHERE contém a expressão `FUNCIONARIO.NDEPTO = DEPARTAMENTO.NUMDEP` que é chamada condição de junção, pois combina duas tuplas: uma de FUNCIONARIO e outra de DEPARTAMENTO, sempre que o atributo NDEPTO de FUNCIONARIO é igual ao atributo NUMDEP de DEPARTAMENTO.

Em SQL, um mesmo nome pode ser usado para dois (ou mais) atributos desde que eles estejam em relações diferentes. Se este é o caso, e a consulta refere-se a dois ou mais atributos com o mesmo nome, deve-se qualificar o nome do atributo com o nome da relação para prevenir ambiguidade (Elmasri e Navathe, 2018). Isso acontece por prefixar o nome da relação com o nome do atributo para separar os dois por um ponto. O exemplo 9 ainda apresenta esta situação por ter o atributo NOME tanto na relação FUNCIONARIO quanto na relação DEPARTAMENTO.

Considere a próxima consulta, listar os projetos em “São Paulo” e os dados do supervisor responsável.

Exemplo 10 – Consulta com junção de três tabelas

```
SELECT PROJETO.NPROJ, DEPARTAMENTO.NOME, FUNCIONARIO.NOME,  
FUNCIONARIO.ENDER  
FROM PROJETO, FUNCIONARIO, DEPARTAMENTO  
WHERE PROJETO.NDEPTO = DEPARTAMENTO.NUMDEP  
AND DEPARTAMENTO.NSUPER = FUNCIONARIO.NUMAT  
AND PROJETO.LOCAL = "São Paulo";
```

No exemplo 10, tem-se duas condições de junção. A primeira é entre PROJETO e DEPARTAMENTO e a segunda ocorre entre DEPARTAMENTO e FUNCIONARIO. Além disso, esse comando SELECT apresenta uma condição de seleção, que filtra todos os projetos cuja localização esteja registrada como São Paulo. Consulta desse tipo com condições de seleção, condições de junção e que resultam em projeção de atributos são conhecidas como consultas seleção-projeção-junção.

O exemplo 11 ilustra um relacionamento recursivo. Deseja-se recuperar os nomes dos funcionários e de seus supervisores imediatos, observe-o.

Exemplo 11 – Consulta com relacionamento recursivo

```
SELECT F.NOME AS NOME_FUNC, S.NOME AS NOME_SUP  
FROM FUNCIONARIO F, FUNCIONARIO S  
WHERE F.NSUPER = S.NUMAT
```

Neste caso, a consulta demonstrada no exemplo 11 usou nomes alternativos para a relação, F e S, chamados *alias* ou apelidos (Elmasri e Navathe, 2018), para a relação FUNCIONARIO. Um *alias* pode seguir o nome da relação ou pode seguir a palavra-chave AS, por exemplo FUNCIONARIO AS F. Também é possível renomear os atributos da relação, como acontece em F.NOME AS NOME_FUNC.

2.3 Tabelas como conjuntos em SQL

SQL não trata uma relação como um conjunto. Tuplas duplicadas podem aparecer mais do que uma vez em uma relação ou no resultado de uma consulta. Mas uma consulta SQL não elimina automaticamente tuplas duplicadas nos resultados da consulta. Para obter apenas tuplas distintas no resultado de uma consulta SQL, utiliza-se a palavra-chave **DISTINCT** na cláusula **SELECT** (Ramakrishnan e Gehrke, 2011). Suponha a consulta na qual deseja-se recuperar o salário de todos os empregados, observe o exemplo 12.

Exemplo 12 – Consulta com valores distintos

```
SELECT DISTINCT SALARIO  
FROM FUNCIONARIO
```

SQL possui alguns operadores de manipulação de conjuntos (Ramakrishnan e Gehrke, 2011) tais como união (**UNION**), intersecção (**INTERSECTION**) e diferença (**EXCEPT**). Considere a consulta que necessite listar projetos que possuem funcionários com sobrenome “Silva”, seja como funcionário ou gerente. O próximo exemplo apresenta uma consulta com o operador **UNION**.

Exemplo 13 – Consulta com operador UNION

```
SELECT NUMPROJ  
FROM PROJETO P, DEPARTAMENTO D, FUNCIONARIO F  
WHERE P.NDEPTO = D.NUMDEP AND  
      D.NSUPER = F.NUMAT AND  
      F.SOBRENOME = 'Silva'  
  
UNION  
  
SELECT NPROJ AS NUMPROJ  
FROM TRABALHA_EM TE, FUNCIONARIO F  
WHERE TE.NUMEMP = F.NUMAT AND  
      F.SOBRENOME = 'Silva'
```

2.4 Consultas aninhadas

Algumas consultas requerem que valores existentes no banco de dados sejam recuperados e usados em uma condição de seleção. Tais consultas podem ser formuladas por meio de consultas aninhadas, como mostra o exemplo 14.

Exemplo 14 – Consultas aninhadas

```
SELECT DISTINCT NUMPROJ
FROM PROJETO P
WHERE NUMPROJ IN (SELECT NUMPROJ
                  FROM PROJETO P, DEPARTAMENTO D, FUNCIONARIO F
                  WHERE P.NDEPTO = D.NUMDEP AND
                      D.NSUPER = F.NUMAT AND
                      F.SOBRENOME = 'Silva')

OR
NUMPROJ IN (SELECT NPROJ
            FROM TRABALHA_EM TE, FUNCIONARIO F
            WHERE TE.NUMEMP = F.NUMAT AND
                  F.SOBRENOME = 'Silva')
```

Segundo Ramakrishnan e Gehrke (2011), uma consulta aninhada é uma consulta que contém outra inserida dentro dela; essa consulta aninhada é chamada de subconsulta (do inglês, *subquery*). O operador IN compara um valor v com um conjunto de valores V e devolve true se v é um dos elementos de V (Elmasri e Navathe, 2018).

2.5 A função EXISTS em SQL

Uma função denominada EXISTS é usada em SQL para verificar se o resultado de uma subconsulta não é vazio, ou seja, se contém alguma tupla, como demonstrado no exemplo seguinte.

Exemplo 15 – Operador EXISTS

```
SELECT F.NOME, F.SOBRENOME  
FROM FUNCIONARIO F  
WHERE EXISTS (SELECT *  
              FROM DEPENDENTE  
              WHERE NUMEMP = F.NUMAT)
```

O exemplo 15 apresentou uma consulta que lista todos os funcionários que possuem dependentes.

2.6 Funções agregadas e agrupamento

Por agregação, entende-se uma operação que toma uma coleção de valores como entrada e devolve um único valor (Silberschatz, Korth e Sudarshan, 2020). Por exemplo, a soma e a média de valores de uma coluna. SQL, não apenas permite agregar colunas, mas também agrupar tuplas de uma relação de acordo com algum critério, como o valor em alguma outra coluna, e, então, agregar dentro de cada grupo. SQL oferece cinco funções agregadas que podem ser aplicadas a uma coluna:

- SUM: a soma dos valores
- AVG: a média dos valores
- MIN: o valor mínimo do conjunto
- MAX: o valor máximo do conjunto
- COUNT: o número de valores (incluindo duplicatas, a menos que elas sejam explicitamente eliminadas com DISTINCT)

Suponha uma consulta em que se deseja obter a soma dos salários, o maior salário, o menor salário e a média salarial de todos os funcionários, como demonstra o exemplo a seguir.

Exemplo 16 – Funções de agregação

```
SELECT SUM(SALARIO), MAX(SALARIO),  
       MIN(SALARIO), AVG(SALARIO)  
FROM FUNCIONARIO
```

Em alguns casos, é útil aplicar as funções agregadas a subgrupos de tuplas de uma relação, com base em alguns valores de atributos. Por exemplo, deseja-se encontrar a média salarial de funcionários por departamento ou o número de empregados que trabalham por departamento.

Exemplo 17 – Funções de agregação com agrupamento

```
SELECT NDEPTO, COUNT(*), AVG(SALARIO)  
FROM FUNCIONARIO  
GROUP BY NDEPTO
```

No exemplo 17, para cada número de departamento, a consulta devolve o número de empregados computado pela função COUNT(*) e a média de salário computada pela função AVG(SALARIO).

Nesses casos, é necessário agrupar as tuplas que possuem o mesmo valor de um ou mais atributos, chamados atributos de agrupamento e, então, aplicar a função de agregação a cada grupo de forma independente. SQL possui a cláusula GROUP BY para este propósito (Elmasri e Navathe, 2018). Esta cláusula especifica o grupo de atributos, que também deveria aparecer na cláusula SELECT, de modo que o valor resultante da aplicação da função a um grupo de tuplas apareça juntamente com o valor do(s) atributo(s) de agrupamento.

Algumas vezes, deseja-se obter os valores de funções agregadas apenas para grupos que satisfazem certas condições. Na consulta a seguir, necessita-se que, para cada projeto em que mais de dois funcionários trabalhem, obtenha-se o número do projeto, o nome do projeto e o número de funcionários que trabalham nele.

Exemplo 18 – Funções de agregação com agrupamento atendendo a uma condição

```
SELECT NUMPROJ, NOMEPROJ, COUNT(*)  
FROM PROJETO P, TRABALHA_EM TE  
WHERE P.NUMPROJ = TE.NPROJ  
GROUP BY NUMPROJ, NOMEPROJ  
HAVING COUNT(*) > 2
```

SQL possui uma cláusula denominada HAVING, que funciona em conjunto com GROUP BY, para este propósito (Elmasri e Navathe, 2018). HAVING impõe uma condição aos grupos de tuplas associados com cada valor de atributo de agrupamento, apenas os grupos que satisfazem a condição são obtidos como resultado da consulta.



PARA SABER MAIS

Para saber mais sobre consultas em SQL, visite o site [W3Schools](https://www.w3schools.com/sql/) (W3Schools, 2025) que apresenta um conjunto completo de instruções e exemplos.

3 Atualizações em SQL

Linguagem de manipulação de dados (do inglês, Data Manipulation Language – DML) são comandos de manipulação de dados, usados para adicionar, excluir e alterar as tuplas da tabela (Silberschatz, Korth e Sudarshan, 2020). Em SQL, há três comandos para modificar a instância do banco de dados: INSERT, DELETE e UPDATE. De maneira resumida, INSERT é usado para adicionar uma tupla à relação. Observe a inserção no exemplo 19.

Exemplo 19 – Comando INSERT

```
INSERT INTO FUNCIONARIO (NOME, SOBRENOME, NUMAT)
VALUES ('JOSE', 'SILVA', 6543)
```

Fonte: adaptado de Elmasri e Navathe (2018).

O comando do exemplo 19 insere uma tupla na tabela FUNCIONARIO. Neste exemplo, o comando informa apenas os valores dos atributos nome, sobrenome e numat. Os outros atributos não informados recebem valores nulos.

Os atributos que não foram especificados no comando de INSERT recebem valores NULL ou DEFAULT. Observe que os valores são listados na mesma ordem em que os seus atributos correspondentes aparecem.

O comando DELETE remove as tuplas da relação (Silberschatz, Korth e Sudarshan, 2020). Ele inclui uma cláusula WHERE semelhante àquela usada em um comando SELECT para consultas. Contudo, no comando DELETE, essa cláusula determina uma condição para selecionar as tuplas que serão removidas.

Exemplo 20 – Comando DELETE

```
DELETE FROM FUNCIONARIO
WHERE NUMDEP IN (SELECT NUMDEP
                  FROM DEPARTAMENTO
                  WHERE NOMEDEP = 'Pesquisa');
```

O comando UPDATE é usado para modificar valores de atributos de uma ou mais tuplas selecionadas (Silberschatz, Korth e Sudarshan, 2020). Como ocorre no comando DELETE, uma cláusula WHERE no comando UPDATE seleciona as tuplas a serem modificadas. Uma cláusula adicional SET especifica os atributos a serem modificados e seus novos valores.

Exemplo 21 – Comando UPDATE

```
UPDATE PROJETO  
SET LOCAL = 'São Paulo'  
WHERE NUMPROJ = 2134;
```

4 Visões em SQL

Uma visão em SQL é uma tabela virtual derivada de outras tabelas. Ela exibe dados de uma ou mais tabelas reais, mas não armazena fisicamente (Elmasri e Navathe, 2018). O comando para especificar uma visão é o CREATE VIEW. A visão recebe um nome, uma lista de nomes de atributos e uma consulta especificando o conteúdo da visão.

Exemplo 22 – Comando CREATE VIEW

```
CREATE VIEW VW_FUNCIONARIO AS  
SELECT NOME, SOBRENOME, ENDER  
FROM FUNCIONARIO
```

Uma visão pode ser descartada, para isso, basta utilizar o comando DROP VIEW. O próximo exemplo utiliza o comando DROP VIEW para eliminar a VIEW VW_FUNCIONARIO.

Exemplo 23 – Comando CREATE VIEW

```
DROP VIEW VW_FUNCIONARIO
```

Considerações finais

Este capítulo apresentou comandos SQL para definição de dados (DDL – *Data Definition Language*), que são comandos usados para estruturar o banco de dados e suas tabelas. Na sequência, o comando SELECT com suas variações. Iniciando desde comando simples para seleção de cam-

pos de uma tabela, até comandos mais elaborados que realizam junção de duas ou mais tabelas, consultas aninhadas e operação de conjuntos.

O texto apresentou também comandos para manipulação de dados (DML – *Data Manipulation Language*), utilizados para inserir, excluir e atualizar os dados em um banco de dados e, por fim, o conceito de visões, que são tabelas virtuais, resultados de consultas do banco de dados.

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

ORACLE. **LiveSQL**. Disponível em: <https://livesql.oracle.com/>. Acesso em: 16 mar. 2025.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistema de gerenciamento de banco de dados**. 3. ed. Porto Alegre: AMGH, 2011.

SILBERSCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

W3Schools. **SQL Operators**. Disponível em: https://www.w3schools.com/sql/sql_operators.asp. Acesso em: 16 mar. 2025.